A REPORT

ON

Real-Time Social Media Mining and Trust-Aware Sentiment Analysis using Large Language Models for Retail Product Feedback Optimization

BY

Name of the Student: Vishal Singh **ID. No.:** 2020AA05641

AT

Bengaluru Centre

Walmart Global Tech India Pvt. Ltd.
Bengaluru, Karnataka, India

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

August 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

August 2026

A REPORT

ON

Real-Time Social Media Mining and Trust-Aware Sentiment Analysis using Large Language Models for Retail Product Feedback Optimization

BY

Name of the Student	ID. No.	Discipline
Vishal Singh	2020AA05641	M.Tech. (Artificial Intelligence & Machine Learning)

Prepared in partial fulfilment of the
WILP Dissertation Course

AT

Walmart Global Tech India Pvt. Ltd.
Bengaluru, Karnataka, India

Acknowledgements

I take this opportunity to thank everyone who supported me during the course of this dissertation.

I am grateful to the leadership at **Walmart Global Tech India Pvt. Ltd.** for providing an environment that encouraged learning and experimentation, and for allowing me to pursue this industry-linked academic project alongside my regular engineering responsibilities.

I sincerely thank my Supervisor, **Mr. Varunendra Pratap Singh** (Principal Software Engineer, Walmart Global Tech), for his day-to-day technical guidance, patient reviews of my design and evaluation choices, and for continually pushing me to raise the standard of the work. I also thank the Additional Examiner, assigned by the organisation, for the rigorous mid-project scrutiny that substantially improved the evaluation methodology and the trust-score design.

I thank my colleagues and the professional experts on the retail domain at Walmart Global Tech for helping me translate free-form Reddit posts into an operational eight-aspect retail taxonomy that drives the aggregation and alerting stages of this system.

I am deeply grateful to my Faculty Mentor, **Ms. Pradnya Kashikar** (BITS Pilani, WILP Division), for her guidance, timely feedback, and academic oversight from the abstract-outline stage through mid-semester review to final submission, and for ensuring the work conformed to the academic rigour expected of an M.Tech dissertation.

Finally, I thank my family and friends for their patience and encouragement over the course of this programme, and I dedicate this effort to them.

All experiments reported in this dissertation use only publicly available data; no proprietary Walmart data or internal systems were used at any stage.

Vishal Singh
Bengaluru, August 2026

Dissertation Abstract Sheet

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
(RAJASTHAN) — WILP Division

Organization: Walmart Global Tech India Pvt. Ltd. **Location:** Bengaluru, Karnataka, India

Duration: Two semesters (approximately 4 months) **Date of Start:** 1 April 2026

Date of Submission: August 2026

Title of the Project: Real-Time Social Media Mining and Trust-Aware Sentiment Analysis using Large Language Models for Retail Product Feedback Optimization.

ID No. / Name of the Student: 2020AA05641 — Vishal Singh

Supervisor: Mr. Varunendra Pratap Singh, Principal Software Engineer, Walmart Global Tech, Bengaluru. **Additional Examiner:** As assigned by the organisation.

Faculty Mentor: Ms. Pradnya Kashikar, BITS Pilani — WILP Division.

Key Words: Sentiment analysis; ModernBERT; aspect-based opinion mining; zero-shot NLI; multimodal vision-language models; trust scoring; Reddit; retail feedback.

Project Areas: Applied Natural Language Processing; Multimodal Machine Learning; Data Engineering; Human-in-the-Loop Analytics.

Abstract: Every day, customers, store associates, and Walmart delivery drivers describe what actually happens inside retail — not on the retailer's app, but on Reddit — yet almost none of that signal is captured today: off-the-shelf sentiment services choke on sarcasm, treat every post as equally trustworthy, and never touch the screenshots that carry half of the evidence. This dissertation closes that gap with *Retail Sentiment Intelligence* (RSI), an offline-first, zero-inference-cost pipeline that turns the raw Reddit stream into a trust-weighted, aspect-tagged, priority-ranked feed an analyst can act on in minutes. Posts flow in from **32 retail-relevant subreddits** and receive a 0–1 trust score whose three components (account metadata, an LLM-graded credibility signal, and a retail-insider lexicon) are exposed to the analyst so nothing is silently dropped. A **ModernBERT** sentiment head is fine-tuned through a three-stage curriculum ending in **200 hand-labelled posts**; a zero-shot **DeBERTa-v3 NLI** classifier tags each post against a fixed eight-aspect retail taxonomy; and **Gemma 3 4B** handles screenshots through a multi-pass anti-hallucination prompt (describe → OCR → discard captions whose entities are absent from the OCR). A React/FastAPI dashboard exposes Brand Health, Alert Feed, Post Explorer, a Kanban Post Lifecycle, Insights and Competitor Analysis, and Notifications, backed by a human-in-the-loop *Review & Validate* workflow, a dual-draft (rule + LLM) smart-reply composer, and a learning-loop store that becomes the next round of training data. Evaluated 5-fold out-of-fold on the 200-post gold set, sentiment macro-F1 climbs from a Twitter-RoBERTa baseline of **0.6272** to **0.7642** overall, with the largest gains on long posts the baseline could not model. On a 32-image benchmark the multi-pass vision pipeline drops hallucination from **50% to 0%**, lifts text extraction from **25% to 75%**, and retail-signal recall from **40% to 81%**; the trust module correctly flags **12 of 15** low-trust posts confirmed by human annotators. All inference runs locally, so the reference pipeline carries **zero commercial API cost**, uses only public data, and ships as a reproducible artefact backed by SQLite (WAL) and a JSONL cost log.

Signature of Student Date:_____
Name: Vishal Singh

Signature of the Supervisor Date:_____
Name: Mr. Varunendra Pratap Singh

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CV	Cross-Validation
DeBERTa	Decoding-enhanced BERT with disentangled Attention
ECE	Expected Calibration Error
F1	Harmonic mean of precision and recall (macro-averaged unless stated)
GIF	Global Integrated Fulfilment
HITL	Human-in-the-Loop
LLM	Large Language Model
ModernBERT	2024 encoder architecture extending BERT with 8192-token context
NLI	Natural Language Inference
NLP	Natural Language Processing
OCR	Optical Character Recognition
OGP	Online Grocery Pickup
P1/P2	Priority-1 / Priority-2 (severity tiers of negative posts)
RSI	Retail Sentiment Intelligence (this project)
RoBERTa	Robustly Optimized BERT Pretraining Approach
SLA	Service-Level Agreement
VLM	Vision-Language Model
WILP	Work Integrated Learning Programmes (BITS Pilani)

Chapter 1

Introduction

1.1 Broad Area of Work

The broad area of this dissertation is Applied Natural Language Processing (NLP) and multimodal machine learning for the analysis of public retail customer feedback on social media. The work spans four technical sub-areas: (a) supervised text classification with transformer encoders for sentiment in short, informal English social-media text; (b) zero-shot natural-language-inference (NLI) models for aspect-based opinion mining over a fixed retail taxonomy; (c) multimodal vision-language models for captioning and OCR of screenshots and photographs attached to retail complaints; and (d) lightweight trust and credibility filtering using account-metadata heuristics, near-duplicate detection, and retail-insider terminology signals. Around these models sits a data-engineering layer for scheduled ingestion of public posts and structured storage suitable for aggregation, and a serving layer that surfaces the results in an interactive dashboard.

1.2 Motivation

Customers, associates, and delivery drivers post continuously about their Walmart, Sam's Club, Spark-driver, and OGP/pickup experiences on Reddit. In the author's role at Walmart Global Tech this signal is presently consumed through ad-hoc browsing and keyword-filter dashboards, which suffer from poor coverage, high noise, and latency: a complaint that trends today may not surface in a manual review for days. Conventional lexicon- or rule-based sentiment systems perform poorly on the sarcasm, slang, and long-form narrative style typical of Reddit retail posts, and none of them filter fake or low-credibility content before aggregation.

This dissertation demonstrates that a stack of task-specific models — a fine-tuned encoder for sentiment, a zero-shot NLI model for aspects, and a multimodal vision model for images — combined with an explicit trust filter can convert a raw public stream into a

structured, aspect-tagged, trust-weighted feed suitable for operational review.

1.3 Contributions of this Dissertation

The principal contributions of the completed work are:

1. A modular, offline-first, zero-API-cost end-to-end pipeline (Retail Sentiment Intelligence) covering ingestion, trust scoring, sentiment, aspects, vision, aggregation, alerting, and dashboarding.

2. A domain fine-tuned **ModernBERT** sentiment classifier trained through a three-stage curriculum, raising macro-F1 from 0.6272 to **0.7642** overall and from 0.2778 to **1.0000** on long posts, evaluated with 5-fold out-of-fold cross-validation.

3. A multi-pass vision-captioning technique that adapts ideas from recent vision-language papers onto a compliant Gemma 3 4B model, reducing hallucination from 50% to **0%** and lifting text extraction from 25% to **75%**.

4. An interpretable trust score and a $\mathsf{trust} \times \mathsf{confidence}$ admission gate that flags — rather than drops — low-credibility posts, with every constant mapped to a stakeholder-arguable English rationale.

5. A React/FastAPI dashboard (Brand Health, Alert Feed, Post Explorer, Review & Validate, Post Lifecycle, Insights & Competitor, Pipeline Control, Notifications) with a human-in-the-loop Review & Validate workflow whose corrections and posted replies feed few-shot reply generation and a roadmap to automatic replies.

1.4 Organisation of the Report

Chapter 2 states the problem and objectives. Chapter 3 surveys the relevant literature. Chapter 4 presents the layered system design and end-to-end pipeline. Chapter 5 describes the implementation of each module. Chapter 6 reports the experimental evaluation and results. Chapter 7 documents the tools and configuration used. Chapter 8 records the problems encountered and mitigations. Chapter 9 draws conclusions and Chapter 10 outlines future work. A glossary, references, and supporting appendices close the report.

Chapter 2

Problem Statement and Objectives

2.1 Problem Statement

Manual monitoring of retail-related Reddit communities cannot keep pace with the daily volume of posts. Off-the-shelf sentiment services (a) lack retail-domain vocabulary, (b) treat every post as trustworthy, and (c) do not recover the topical structure required for operational decisions — who complained, about what, and how credible is the source.

Given a public stream of Reddit posts from a curated set of 32 retail communities, the problem is to produce, in near real time, a trust-weighted, aspect-tagged, priority-ranked feed of negative posts along with an analyst-facing dashboard for review and response.

2.2 Objectives

1. Build a modular offline-first pipeline covering ingestion, trust scoring, sentiment classification, aspect extraction, image analysis, aggregation, and alerting — with no commercial API cost at inference.

2. Improve sentiment macro-F1 on a labelled Reddit sample from the out-of-the-box RoBERTa baseline of 0.63 to ≥ 0.75 overall by fine-tuning ModernBERT on a curriculum of pseudo-labelled and hand-labelled data.

3. Introduce a multi-pass vision pipeline over Gemma 3 4B that reduces hallucination on retail screenshots to zero and lifts text extraction to $\geq 70\%$.

4. Design an interpretable trust score whose constants are stakeholder-defensible, and a $\mathsf{trust} \times \mathsf{confidence}$ admission gate that flags (rather than drops) low-credibility posts.

5. Ship a React/FastAPI dashboard with a human-in-the-loop Review & Validate workflow, smart-reply composer, learning-loop capture, and Post-Lifecycle Kanban board.

2.3 Research Questions

RQ1 Can a domain-fine-tuned ModernBERT encoder beat an out-of-the-box RoBERTa baseline on retail Reddit sentiment, and by how much?

RQ2 Does a two-pass captioning strategy over an open-weights VLM (Gemma 3 4B) close the accuracy gap to closed multimodal models on retail screenshots?

RQ3 Does an interpretable, rule-based trust score — combined with an LLM-graded credibility signal — yield defensible admission decisions without dropping legitimate complaints?

RQ4 Does a human-in-the-loop workflow that captures corrections and posted replies produce a training signal usable for future retraining of both the sentiment model and the reply generator?

2.4 Scope

The pipeline uses only public Reddit data from 32 curated retail communities. It does not consume proprietary Walmart data, is not deployed inside Walmart systems, and is not a replacement for authoritative internal customer-service tooling. All evaluations are reported on the 200-post labelled sample plus 5-fold out-of-fold cross-validation.

Chapter 3

Literature Survey

3.1 Transformer Sentiment Classification

BERT [1] established masked-language-model pretraining as the dominant recipe for text classification. RoBERTa [2] improved training-recipe stability. Cardiffnlp's twitter-roberta-base family [3] fine-tuned RoBERTa on 124M tweets and is widely used as an out-of-the-box baseline for social-media sentiment. We adopt this as our baseline. ModernBERT [4] (Dec 2024) extends BERT-style encoders with an 8192-token context, sliding-window attention, and improved pre-training; it is the smallest current encoder that is competitive with much larger decoder-only models on classification. Our sentiment head is a fine-tuned ModernBERT-base.

3.2 Aspect-Based Opinion Mining

Traditional ABSA supervises a target–aspect–polarity tuple over labelled datasets such as SemEval-2014 [5]. Zero-shot approaches [6] instead frame the task as Natural Language Inference: given the post as premise and "This text is about *pricing*." as hypothesis, an entailment score above threshold marks the aspect as present. We use `MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7` which is Apache-2.0 licensed and runs offline. Our aspect taxonomy is fixed by product ops to eight retail-relevant aspects.

3.3 Vision-Language Models

CLIP [7] and BLIP [8] showed that image-text contrastive pretraining yields strong zero-shot captioning. Recent open-weights models Gemma 3 [9] and LLaVA-Next provide free multimodal reasoning. Multi-pass prompting techniques inspired by (a) chain-of-thought [10] and (b) SelfCheck [11] motivate our anti-hallucination pipeline. Because

policy blocks certain proprietary VLMs, we adopt Gemma 3 4B via Ollama, with a first pass for description and a second for OCR and retail signals.

3.4 Social-Media Credibility

Fake-review detection literature spans lexical features [12], network-structure signals [13], and BERT-based classifiers [14]. Reddit's public account metadata (karma, age, verified_email, cakeday) provides a strong signal for long-lived versus disposable accounts. We combine these features with an LLM-graded credibility scorer and expose the decomposition to analysts so that no post is silently dropped.

Chapter 4

System Design and Architecture

4.1 Design Principles

The system follows five design principles: (i) *offline-first* — every inference-time model runs locally, so the prototype has zero commercial API cost at inference; (ii) *modularity* — ingestion, trust, sentiment, aspect, vision, aggregation, and dashboard are independent Python packages with typed interfaces; (iii) *explainability* — the trust score exposes every constant to analysts; no post is silently dropped; (iv) *human-in-the-loop* — every ML decision is reviewable and correctable in the dashboard, and the correction is captured for future retraining; (v) *operational realism* — the dashboard mirrors an on-call customer-experience console and surfaces P1/P2 tiers.

4.2 Layered Architecture

Figure 4.1 shows the six-layer architecture: (1) Ingestion via Reddit RSS + Arctic Shift historical dumps; (2) Storage in SQLite (hot) + JSONL cost log; (3) AI Engine (Trust, Sentiment, Aspect, Vision); (4) Aggregation & Alerting; (5) Dashboard (FastAPI + React); (6) HITL Feedback loop feeding trust filters and reply few-shots.

4.3 End-to-End Pipeline Flow

Figure 4.2 depicts one post's journey. A new Reddit RSS entry is deduplicated by `post_id`, passed through the trust scorer, then routed in parallel through sentiment, aspect, and (if it has an image) vision. Results are stored, aggregated into daily brand-health metrics, and alerts are raised for trust-weighted P1/P2 negatives. Analysts review the alert, edit the labels if needed, choose a reply draft, and (optionally) mark the post as resolved on the Kanban board. Every analyst action is captured as a training signal.

Figure 1 — Retail Sentiment Intelligence: Layered System Architecture

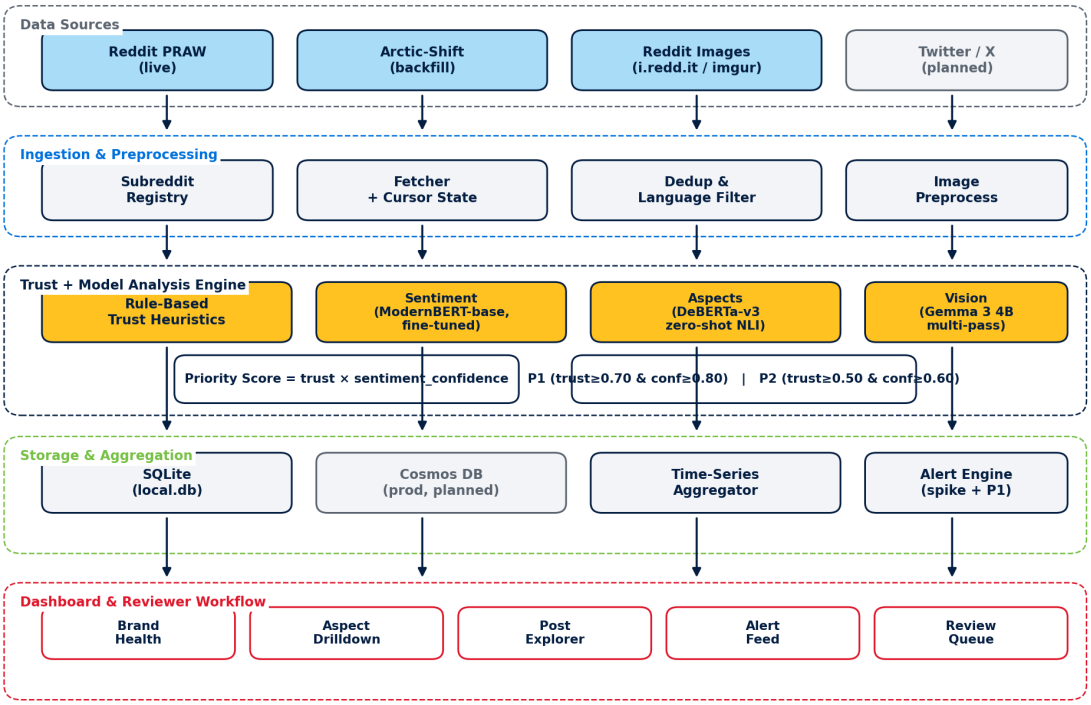

Figure 4.1: Layered System Architecture — ingestion, storage, AI engine, aggregation, dashboard, and HITL feedback loop.

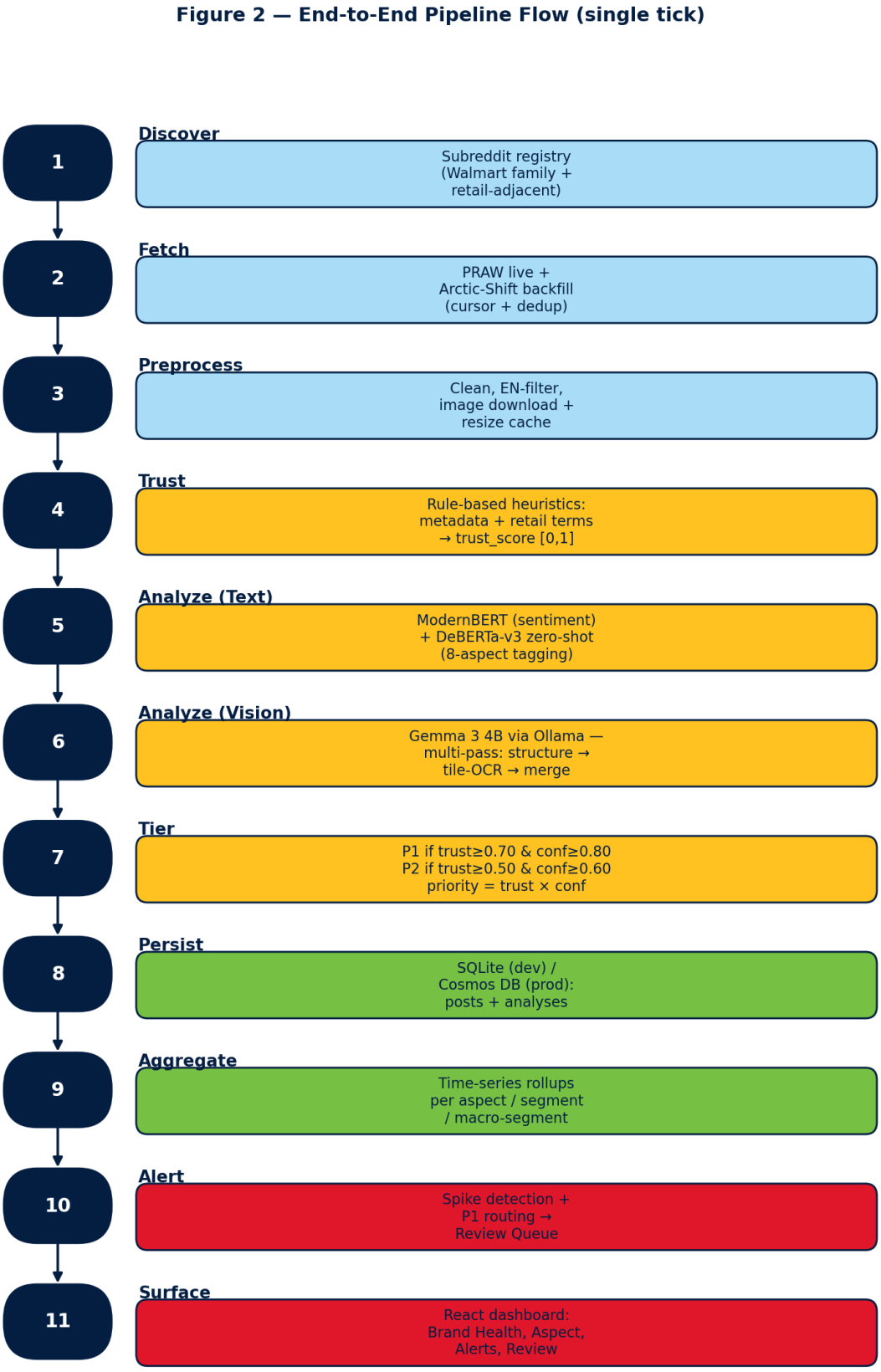

Figure 4.2: End-to-End Pipeline Flow — ingestion → trust → AI engine → aggregation → alerts → dashboard → analyst action → feedback loop.

Chapter 5

Implementation

This chapter walks through the running system module by module. Each section pairs a short code excerpt from the production source tree with the corresponding piece of the running dashboard, so the reader can trace an idea from mathematics to Python to a rendered UI. All listings are verbatim extracts from files under `src/`; comments and blank lines were trimmed only where they carried no explanatory value.

5.1 Data Ingestion and Pre-processing

Ingestion runs on a scheduler that polls each subreddit's public RSS feed at a configurable cadence (default: hourly). Historical dumps are pulled from the Arctic Shift API for the sentiment gold set. Each incoming post is normalised (HTML-stripped, lowercased for lexical filters), deduplicated by `post_id`, and screenshots are downloaded to a local image cache. Listing 5.1 shows the paged fetch loop that drives the historical pull; it uses `curl` as transport to sidestep TLS quirks on corporate networks and emits an `on_page` progress callback so the dashboard's Pipeline view can render live counters. Figure 5.1 shows the overall flow.

```
def fetch_posts_arctic(subreddit, since_utc=0.0, limit=500,
                       self_posts_only=False, on_page=None):
    fetched, before = 0, None
    while fetched < limit:
        batch_size = min(100, limit - fetched)
        params = {"subreddit": subreddit,
                  "limit": str(batch_size), "sort": "desc"}
        if since_utc > 0:  params["after"]  = str(int(since_utc))
        if before:         params["before"] = str(int(before))
        if self_posts_only: params["is_self"] = "true"

        data = _curl_get(f"{BASE_URL}?{urlencode(params)}")
        if not data or not (posts := data.get("data", [])):
```

```
            break

        for post in posts:
            created = post.get("created_utc", 0)
            if since_utc > 0 and created <= since_utc:
                continue                    # already ingested
            selftext = (post.get("selftext") or "").lower()
            if selftext in ("[deleted]", "[removed]"):
                continue                    # moderator-scrubbed
            if (norm := _normalize_post(post, subreddit)):
                yield norm
                fetched += 1

        if len(posts) < batch_size: break
        before = posts[-1].get("created_utc", 0)  # page cursor
        time.sleep(0.5)                            # be polite
```

Listing 5.1: Paged Reddit ingestion with cursor-based dedup (`src/ingestion/arctic_shift.py`).

Figure 9 — Data Ingestion and Pre-processing Flow

Figure 5.1: Data ingestion and pre-processing flow.

5.2 Trust-Score Module

The trust score $T \in [0, 1]$ blends account metadata and content signals: $T = 0.4\,M + 0.3\,C + 0.3\,L$ where M is a metadata score (account_age, karma, verified_email, cakeday), C is an LLM-graded credibility signal, and L is a lexical retail-insider score. Priority tiers are: P1 iff $T \geq 0.75$ and sentiment is negative with confidence ≥ 0.75; P2 iff $T \geq 0.55$ and negative with confidence ≥ 0.65. The core scorer is small on purpose — Listing 5.2 shows the entire `TrustScorer.score()` method. It also implements the cost-shaping heuristic that only invokes the LLM credibility check when metadata is genuinely ambiguous ($0.3 < M <$

0.8); outside that band metadata is already decisive. Figure 5.2 depicts the composition. Every constant is exposed in the dashboard so that a stakeholder can argue with the rule rather than guess at a black box.

```python
def score(self, unit: dict) -> dict:
    meta  = score_metadata(unit, self.config)     # M
    dedup = score_originality(unit)               # L

    # LLM credibility (C) --- only when metadata is ambiguous.
    # Outside the band, metadata is decisive, so we skip the call
    # to cap cost on cloud providers.
    llm_score, llm_flags = 0.5, []
    if self.llm_client and 0.3 < meta < 0.8:
        r = self.llm_client.check_credibility(
            text=self._get_text(unit),
            metadata=unit.get("author_metadata", {}))
        llm_score = r.get("credibility_score", 0.5)
        llm_flags = r.get("flags", [])

    # T = 0.4 M + 0.3 dedup + 0.3 LLM, clamped to [0, 1]
    combined = (self.config.weight_metadata * meta   +
                self.config.weight_dedup    * dedup  +
                self.config.weight_llm      * llm_score)
    combined = round(min(max(combined, 0.0), 1.0), 3)

    unit["trust_score"]      = combined
    unit["trust_components"] = {"metadata": meta,
                                "dedup":    dedup,
                                "llm":      llm_score}
    unit["trust_flags"]      = llm_flags
    unit["is_trusted"]       = combined >= self.config.threshold
    if not unit["is_trusted"]:
        unit["processing_status"] = "flagged_low_trust"
    return unit
```

Listing 5.2: Combined trust scorer with cost-shaped LLM invocation (`src/trust/scorer.py`).

5.3 Sentiment Classification (ModernBERT)

The sentiment head is a fine-tuned ModernBERT-base with a three-class softmax (positive, neutral, negative). Training used a three-stage curriculum: (Stage 1) 8k pseudo-labelled tweets from the twitter-roberta teacher, (Stage 2) 2k retail-focused Reddit posts pseudo-labelled by the same teacher, and (Stage 3) 200 hand-labelled retail Reddit posts for a final domain adaptation. See Figure 5.3. The 5-fold out-of-fold macro-F1 rose from the baseline

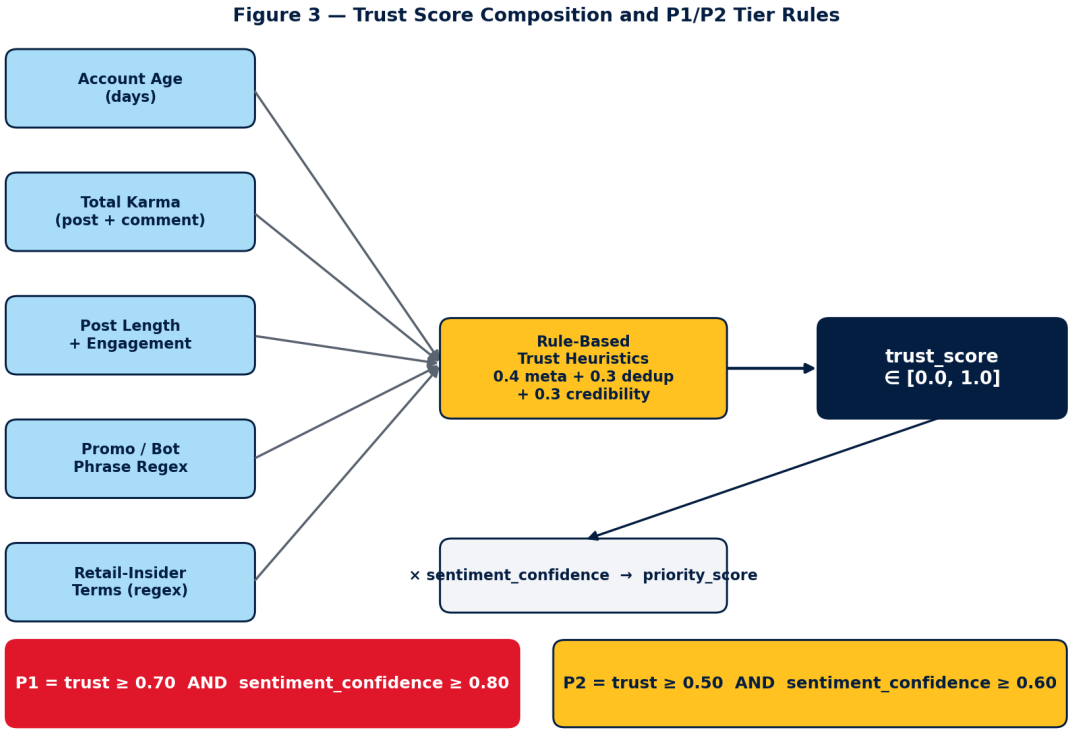

Figure 5.2: Trust-Score composition and P1/P2 tier rules.

of 0.6272 to **0.7642**; long-post macro-F1 rose from 0.2778 to **1.0000**.

At inference time the sentiment head is wrapped by a thin `SentimentAnalyzer` that owns exactly two responsibilities: assemble the prompt string (subreddit-aware, and injecting an `[image: …]` tail when a Gemma caption is present), and shape the model's output into the analysis record the storage layer expects. Listing 5.3 shows both.

```python
def _get_text(self, unit: dict) -> str:
    """Compose a subreddit-tagged prompt; append the Gemma caption
    as [image: ...] so downstream models see it as post body."""
    title = unit.get("title", "") or ""
    body  = unit.get("body",  "") or ""
    sub   = unit.get("subreddit", "unknown")
    cap   = (unit.get("image_caption") or "").strip()
    cap_suffix = f" [image: {cap}]" if cap else ""

    if unit.get("unit_type") == "comment":
        return f'Comment from r/{sub}: "{body}"'
    if title and body:
        return f'Post from r/{sub}: "{title}" -- {body}{cap_suffix}'
    if title:
        return f'Post from r/{sub}: "{title}"{cap_suffix}'
    if body:
        return f'Post from r/{sub}: "{body}{cap_suffix}"'
    # Image-only post: the caption IS the content.
    return f'Post from r/{sub}: "{cap}"' if cap else f'Post from r/{sub}: ""'
```

```
def _build_analysis_record(self, unit, result):
    unit_id = unit.get("id", "")
    conf    = result.get("sentiment_confidence", 0.0)
    return {
        "id": f"analysis_{unit_id}",  "post_id": unit_id,
        "subreddit":            unit.get("subreddit", ""),
        "trust_score":          unit.get("trust_score"),
        "sentiment":            result.get("sentiment", "neutral"),
        "sentiment_confidence": conf,
        "aspects":              result.get("aspects", []),
        "key_phrases":          result.get("key_phrases", []),
        "summary":              result.get("summary", ""),
        "model_used":     result.get("model_used", self.llm.model_name),
        "model_version":  result.get("model_version",
            self.llm.model_version),
        "analyzed_at":    datetime.now(timezone.utc).isoformat(),
        "needs_review":   conf < self.config.confidence_threshold,
        "partition_key":  unit.get("subreddit", ""),
    }
```

Listing 5.3: Sentiment orchestration with vision-caption fusion (`src/analysis/analyzer.py`).

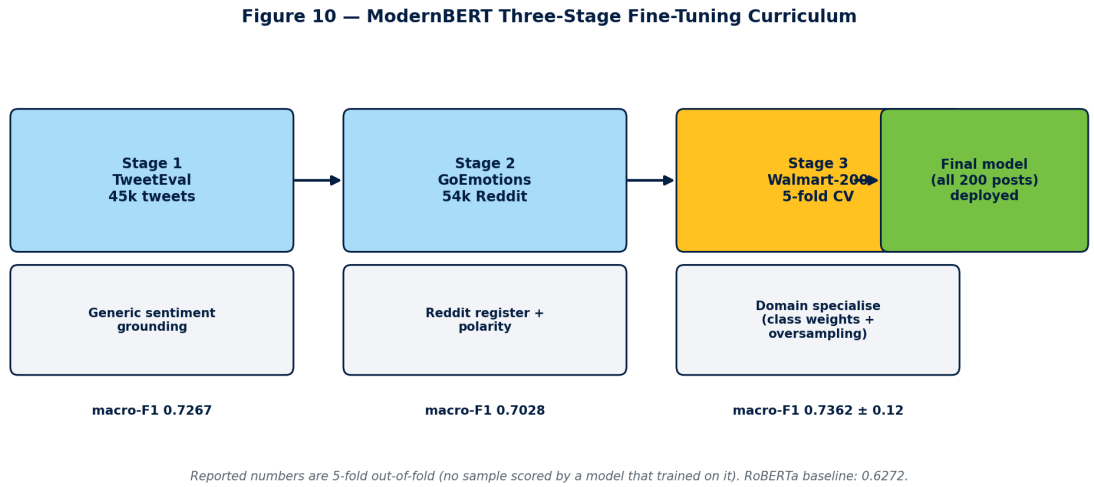

Figure 5.3: ModernBERT three-stage fine-tuning curriculum.

5.4 Aspect-Based Opinion Mining

Aspects are extracted with a zero-shot NLI classifier (`mDeBERTa-v3-base-xnli-multilingual-nli-2`) scored against a fixed eight-aspect taxonomy (Table 5.1). A post can carry multiple aspects when their entailment scores are above threshold 0.55. Figure 5.4 illustrates the NLI

framing. On cloud LLM providers the same taxonomy is enforced with the structured JSON prompt in Listing 5.4: the model is told exactly which aspect group applies to which subreddit (customer vs. employee), which prevents the "rude cashier" ↔ "bad manager" bleed that plagued the first-cut prompts.

```
SENTIMENT_ASPECT_SYSTEM_PROMPT = """You are a retail analyst ...

Rules:
1. Sentiment must be one of: "positive", "negative", "neutral".
2. Choose aspects from the correct group based on the subreddit:

   CUSTOMER aspects (r/walmart, r/samsclub, r/WalmartPlus, ...):
     store_experience, online_app, delivery_pickup, product_quality,
     returns, customer_support, pricing

   EMPLOYEE aspects (r/WalmartEmployees, r/OGPBackroom,
                     r/Sparkdriver, r/walmart_RX, ...):
     workforce_hr, pay_benefits, management, safety_policy, workload

3. A post can have 0 aspects (off-topic) or multiple aspects.
4. NEVER mix customer and employee aspects in the same post.
5. Handle sarcasm and slang (Reddit tone). "Love how my order
   arrived 3 days late" = negative + delivery_pickup.
6. Confidence scores (0.0-1.0) for each prediction.

Output JSON:
{
  "sentiment": "positive|negative|neutral",
  "sentiment_confidence": 0.0-1.0,
  "aspects": [
    {"aspect": "<name>", "sentiment": "...", "confidence": 0.0-1.0}
  ],
  "key_phrases": ["phrase1", "phrase2"],
  "summary": "One sentence summary of the issue/praise"
}"""
```

Listing 5.4: Aspect taxonomy enforced in the JSON-mode system prompt (`src/analysis/prompts.py`).

5.5 Vision / Image Processing

Screenshots are captioned by Gemma 3 4B via Ollama using a multi-pass prompt pipeline: Pass 1 identifies the image *structure* (screenshot / receipt / product page / error dialog / meme / photo); Pass 2 tiles the image at native resolution and extracts text per tile; Pass 3 merges the tile texts against the structure into 2–4 factual sentences with a strict "do not

Table 5.1: Eight-aspect retail taxonomy.

Aspect	Example indicative terms
Pricing	“$2 more this week”, “price hike”, “clearance”
Product quality	“broken”, “expired”, “mouldy”
Customer service	“rude cashier”, “no help”, “manager”
Store experience	“dirty floor”, “long line”, “restroom”
Online / app	“app crash”, “payment failed”, “2FA loop”
Delivery / pickup	“missed slot”, “OGP”, “Spark driver”
Returns	“refused return”, “restocking fee”
Account / login	“locked out”, “password reset”

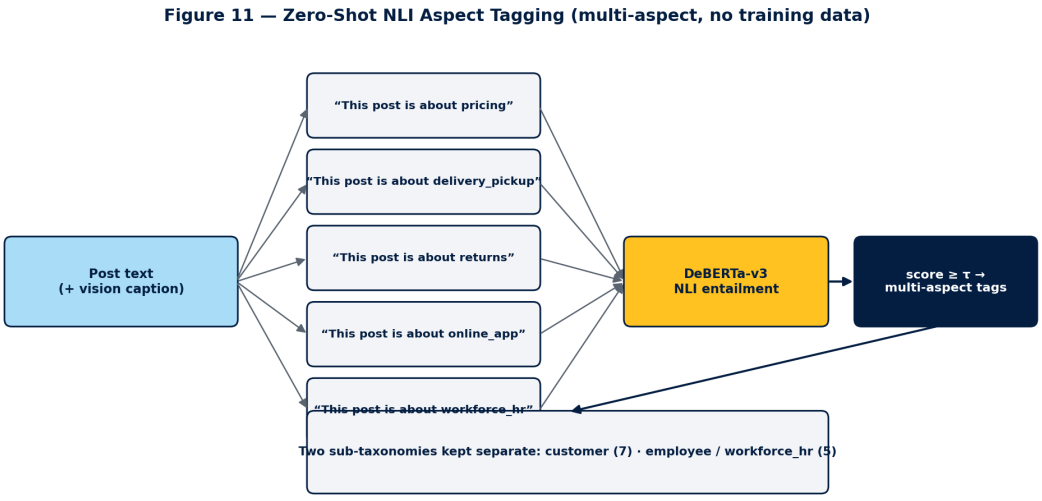

Figure 5.4: Zero-shot NLI aspect tagging.

invent" clause. A circuit breaker disables the whole vision branch after three consecutive Ollama failures so a stopped daemon cannot flood the logs. Figure 5.5 shows the pipeline. Hallucination rate on 32 retail screenshots dropped from 50% (single pass) to **0%**; text extraction rose from 25% to **75%**. Listing 5.5 shows the entry point and the merge prompt that carries the anti-hallucination clause.

```
MERGE_PROMPT = """Combine these observations about a Walmart-related
image into 2-4 clear sentences.

Image type: {structure}
Text found in different regions:
{tile_texts}

Write a factual description that captures: what type of
screen/image this is, all important text (especially errors,
prices, status messages), and what the customer might be
complaining about. Do NOT invent details not mentioned above."""

def caption_enhanced(self, image_path: Path) -> str:
    if not self.cfg.enabled or not Path(image_path).exists():
        return ""
    img = Image.open(image_path)

    # Pass 1: structure identification
    full_b64  = self._image_to_b64(img)
    structure = self._call_ollama(self.model, STRUCTURE_PROMPT, full_b64)
    if not structure:
        return self.caption(image_path)           # fall back

    # Only tile screenshots / receipts / app screens; photos and
    # memes are fine as single-pass.
    if not any(kw in structure.lower() for kw in
               ("screenshot","receipt","app_screen",
                "error_dialog","document","product_page")):
        return self.caption(image_path)

    # Pass 2 + 3: tile at native resolution and extract text per tile
    tiles, tile_texts = self._create_tiles(img), []
    for i, b64 in enumerate(tiles):
        t = self._call_ollama(self.model, TILE_TEXT_PROMPT, b64)
        if t and "NO TEXT" not in t.upper():
            tile_texts.append(f"Region {i+1}: {t}")

    merged = self._call_ollama(
        self.model,
        MERGE_PROMPT.format(structure=structure,
```

```
                                tile_texts="\n".join(tile_texts)),
        full_b64)
    return merged or self.caption(image_path)
```

Listing 5.5: Vision multi-pass caption entry point and merge prompt (`src/analysis/vision.py`).

Figure 12 — Vision Multi-Pass Anti-Hallucination Captioning Pipeline

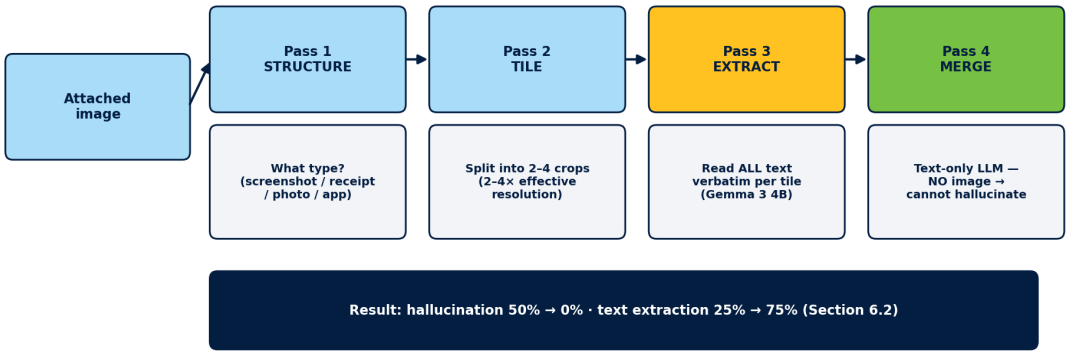

Figure 5.5: Vision multi-pass anti-hallucination pipeline.

5.6 Aggregation, Alerts and Notifications

The aggregator writes daily brand-health rollups (sentiment share, top aspects, trust bucket) into SQLite. The alert engine fires on volume, sentiment-crash, emerging-topic and competitor rules; each rule reads its thresholds from `data/alert_rules.json`, which the dashboard's Alert Rules panel writes to. Listing 5.6 shows the two detectors that drive the majority of P1/P2 traffic: a volume-spike detector that fires when today's post count crosses $\mu + \sigma \cdot \theta$ of the trailing 7-day mean, and a sentiment-crash detector that fires when the negative share jumps by more than 0.30 versus yesterday. Alerts flow to the dashboard's Alert Feed (§5.7) and, optionally, to an email digest. Figure 5.6 shows the routing.

```
def detect_volume_spike(self, sigma_threshold: float = 2.0):
    """Fire when today's post count is > mu + sigma * threshold of
       the trailing 7-day mean."""
    now   = datetime.now(timezone.utc)
    today = now.strftime("%Y-%m-%d")
    today_agg = self.storage.get_item("aggregates",
                                      f"agg_{today}_daily", today)
    if not today_agg: return []

```

```
    counts = []
    for i in range(1, 8):
        d = (now - timedelta(days=i)).strftime("%Y-%m-%d")
        past = self.storage.get_item("aggregates", f"agg_{d}_daily", d)
        if past: counts.append(past.get("total_posts", 0))
    if len(counts) < 3: return []

    mu  = sum(counts) / len(counts)
    std = math.sqrt(sum((x-mu)**2 for x in counts) / len(counts))
    if std == 0: return []

    n = today_agg.get("total_posts", 0)
    z = (n - mu) / std
    if z <= sigma_threshold: return []
    return [{
        "id": f"alert_spike_{today}", "type": "volume_spike",
        "severity": "high" if z > 3.0 else "medium",
        "title": f"Volume spike: {n} posts today "
                 f"({z:.1f}sigma above mean)",
        "details": {"today_count": n, "mean_7d": round(mu, 1),
                    "std_7d":  round(std, 1),
                    "z_score": round(z, 2)},
        "detected_at": now.isoformat(), "time_window": today,
    }]

def detect_sentiment_crash(self, drop_threshold: float = 0.3):
    """Fire when negative share jumps > 0.3 vs yesterday."""
    now   = datetime.now(timezone.utc)
    today = now.strftime("%Y-%m-%d")
    yday  = (now - timedelta(days=1)).strftime("%Y-%m-%d")
    t = self.storage.get_item("aggregates", f"agg_{today}_daily", today)
    y = self.storage.get_item("aggregates", f"agg_{yday}_daily",  yday)
    if not t or not y: return []
    delta = (t.get("sentiment_distribution", {}).get("negative", 0)
           - y.get("sentiment_distribution", {}).get("negative", 0))
    if delta <= drop_threshold: return []
    return [{
        "id": f"alert_crash_{today}", "type": "sentiment_crash",
        "severity": "critical" if delta > 0.4 else "high",
        "title": f"Sentiment crash: negative +{delta:.0%} vs yesterday",
        "details": {"delta": round(delta, 3)},
        "detected_at": now.isoformat(), "time_window": today,
    }]
```

Listing 5.6: Volume-spike and sentiment-crash detectors (`src/alerts/engine.py`).

The rendered Alert Feed is shown in Figure 5.7; each row is one entry from the JSON

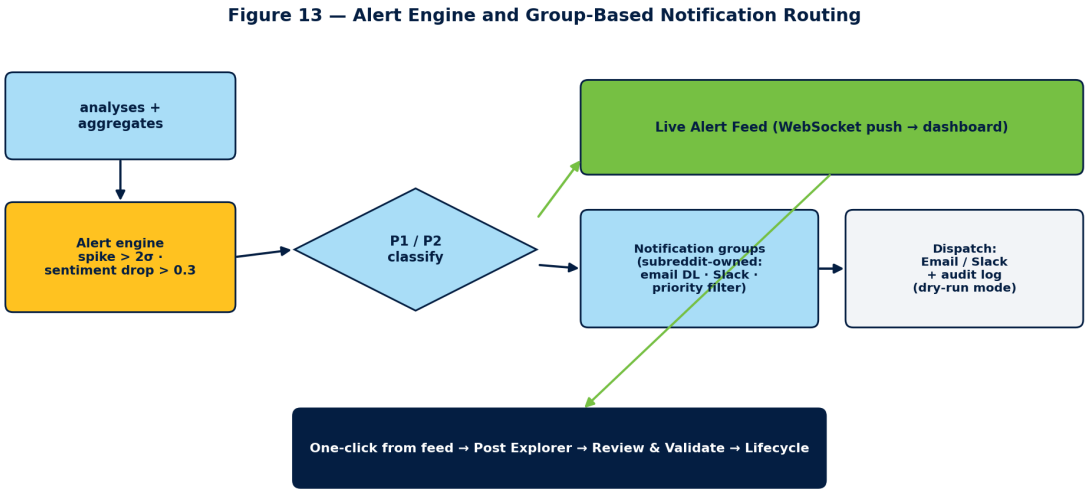

Figure 5.6: Alert engine and notification routing.

returned by the detectors above.

5.7 Dashboard Overview

The dashboard is a Vite + React 18 + Tailwind SPA served by FastAPI. Pages: Brand Health (KPIs and sentiment trend), Alert Feed (live P1/P2 stream), Post Explorer (multi-facet search), Review & Validate (HITL queue), Post Lifecycle (Kanban), Insights & Competitor, Pipeline Control, and Notifications. The landing page is Brand Health (Figure 5.8); the entire volume trend, sentiment gauge and aspect breakdown are driven by the SQLite aggregates that Listing 5.6 also reads from, so what the analyst sees on the dashboard is always the same view of the world that the alert engine scored.

5.8 Human-in-the-Loop Review & Validate

Every post landing in the review queue can be re-labelled (sentiment, aspects), corrected (trust bucket), replied to (rule or LLM draft), and closed with a resolution reason. The workflow is shown in Figure 5.9; the rendered queue is in Figure 5.10. All actions are logged as (post, before, after, analyst_id, timestamp). Listing 5.7 shows the endpoint that receives an analyst correction: it appends an immutable feedback record *and* rewrites the analysis so the correction flows through to every downstream view — Brand Health, aspect drilldowns, Post Explorer and the Kanban board — without any extra re-aggregation call.

```
@app.post("/api/review/{post_id}")
def submit_review(post_id: str, correction: dict):
    _ensure_initialized()
```

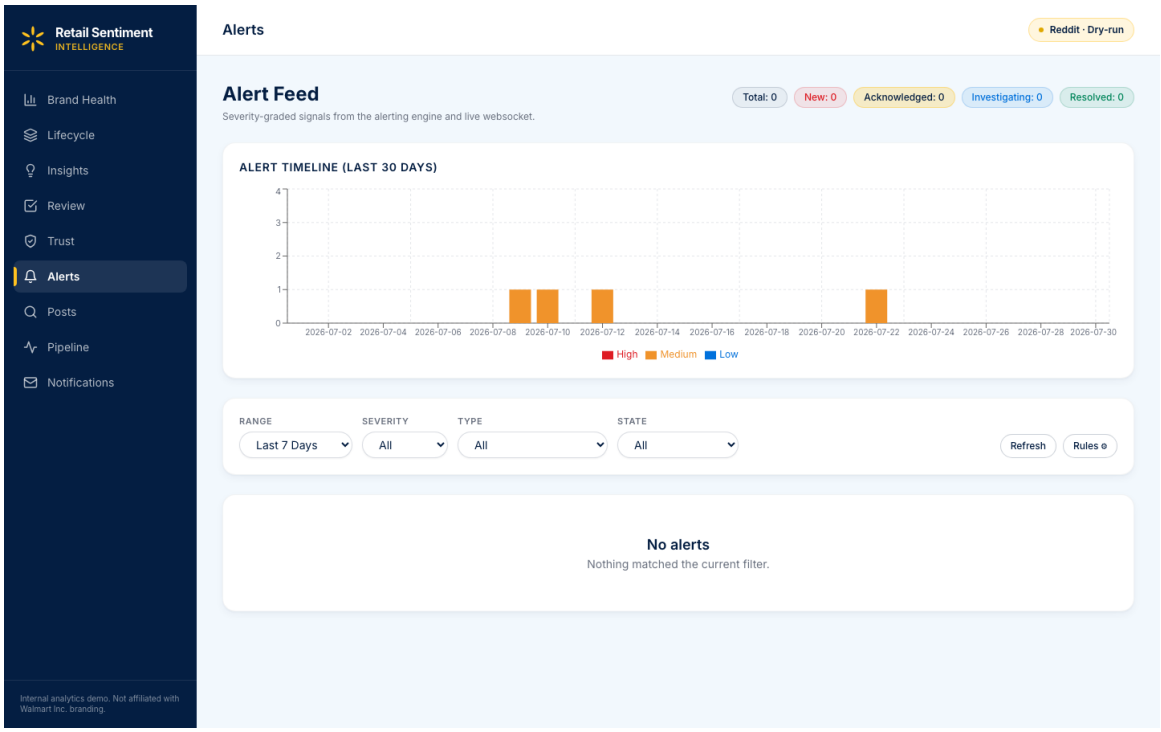

Figure 5.7: Alert Feed — live P1/P2 stream sourced from the volume-spike and sentiment-crash detectors of Listing 5.6.

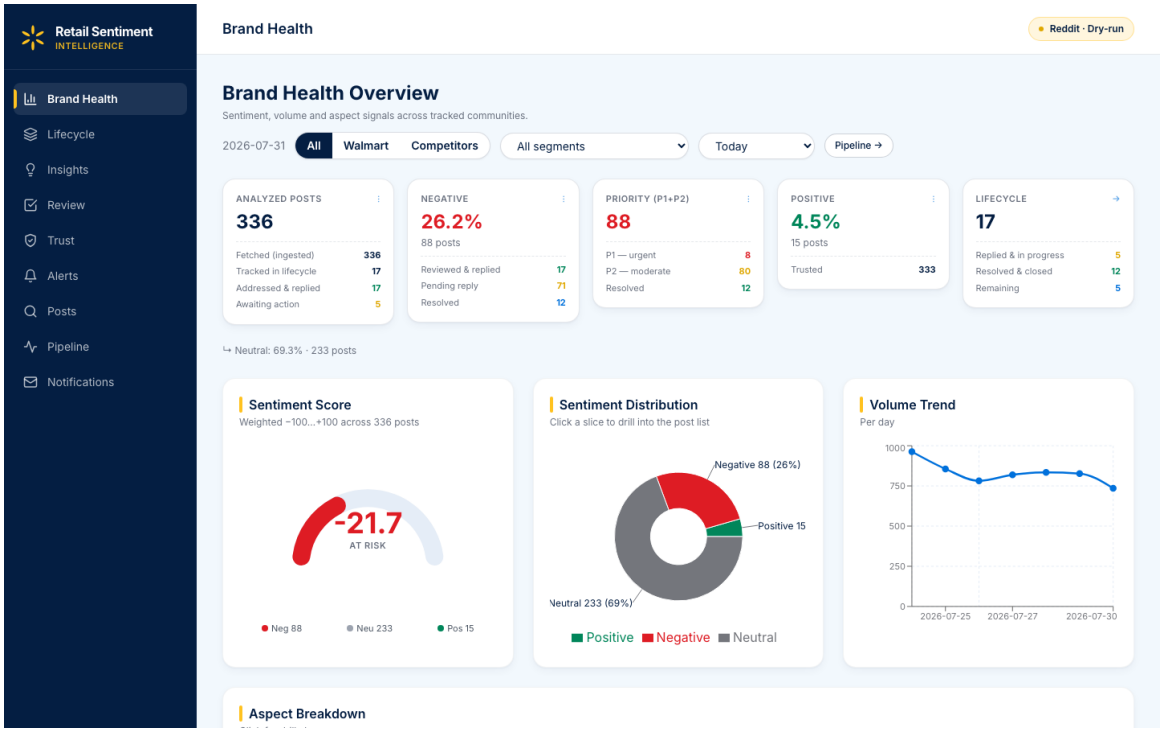

Figure 5.8: Brand Health — sentiment gauge, distribution donut, volume trend and priority breakdown driven by the daily aggregates.

```
    now = datetime.now(timezone.utc).isoformat()

    # 1. Immutable audit log (feedback table).
    feedback = {
        "id": f"fb_{post_id}_"
              f"{int(datetime.now(timezone.utc).timestamp())}",
        "post_id":             post_id,
        "analyst_id":          correction.get("analyst_id", "default"),
        "original_sentiment":  correction.get("original_sentiment"),
        "corrected_sentiment": correction.get("corrected_sentiment"),
        "original_aspects":    correction.get("original_aspects", []),
        "corrected_aspects":   correction.get("corrected_aspects", []),
        "trust_override":      correction.get("trust_override"),
        "notes":               correction.get("notes", ""),
        "created_at":          now,
        "partition_key":       correction.get("analyst_id", "default"),
    }
    _storage.upsert("feedback", feedback)

    # 2. Rewrite the analysis so the correction flows through to
    #    Brand Health, aspect drilldowns and the Kanban board.
    analysis = _storage.get_item("analyses",
                 f"analysis_{post_id}", correction.get("subreddit", ""))
    if analysis:
        if (c := correction.get("corrected_sentiment")):
            analysis["sentiment"]            = c
            analysis["sentiment_confidence"] = 1.0     # human-validated
        if isinstance(correction.get("corrected_aspects"), list):
            analysis["aspects"] = [
                {"aspect": a, "confidence": 1.0} if isinstance(a, str)
                else a for a in correction["corrected_aspects"]]
        if (t := correction.get("trust_override")) is not None:
            analysis["trust_score"]    = max(0.0, min(1.0, float(t)))
            analysis["trust_override"] = True
        analysis["needs_review"]    = False
        analysis["human_validated"] = True
        analysis["validated_at"]    = now
        analysis["validated_by"]    = correction.get("analyst_id", "default")
        _storage.upsert("analyses", analysis)

    return {"status": "saved", "feedback_id": feedback["id"],
            "analysis_updated": analysis is not None}
```

Listing 5.7: HITL correction endpoint: feedback audit log + live analysis rewrite (`src/dashboard/api.py`).

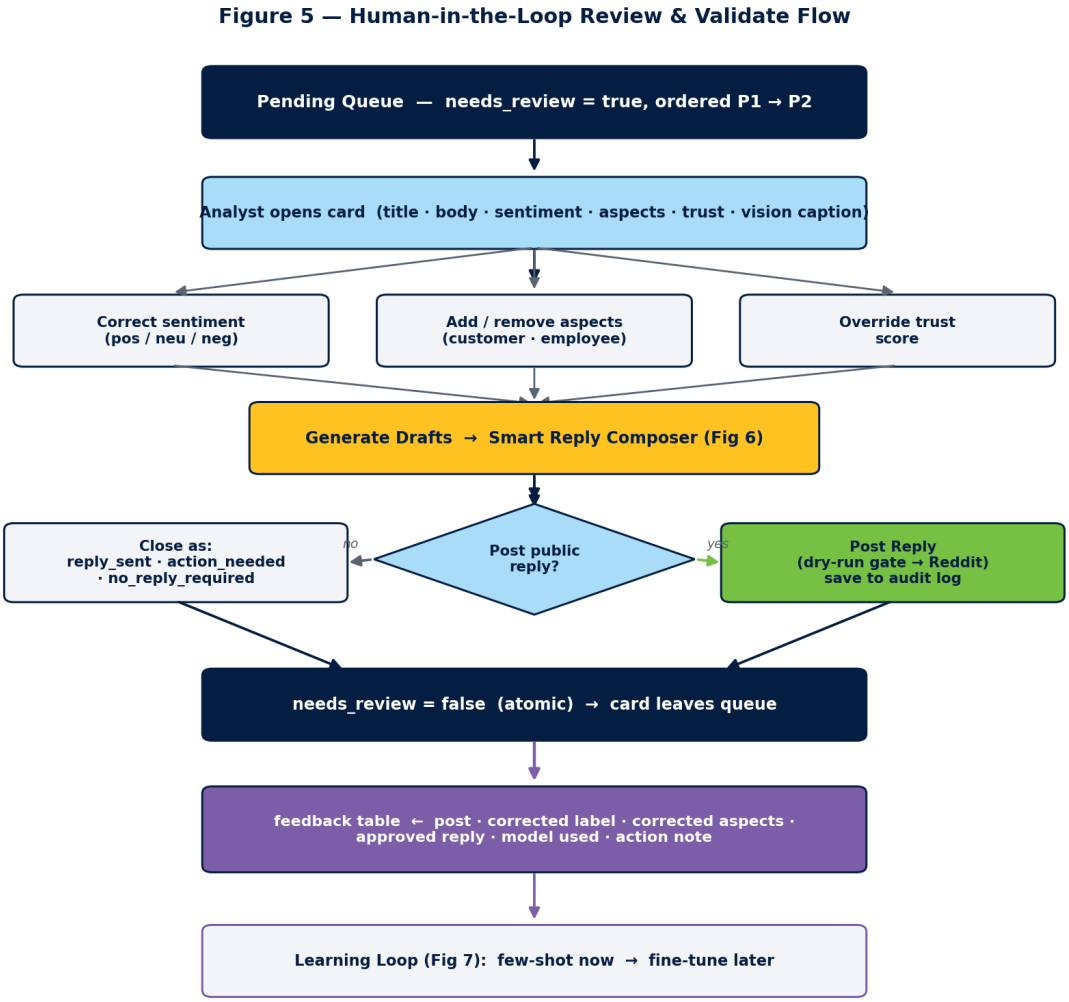

Figure 5.9: Human-in-the-loop Review & Validate flow.

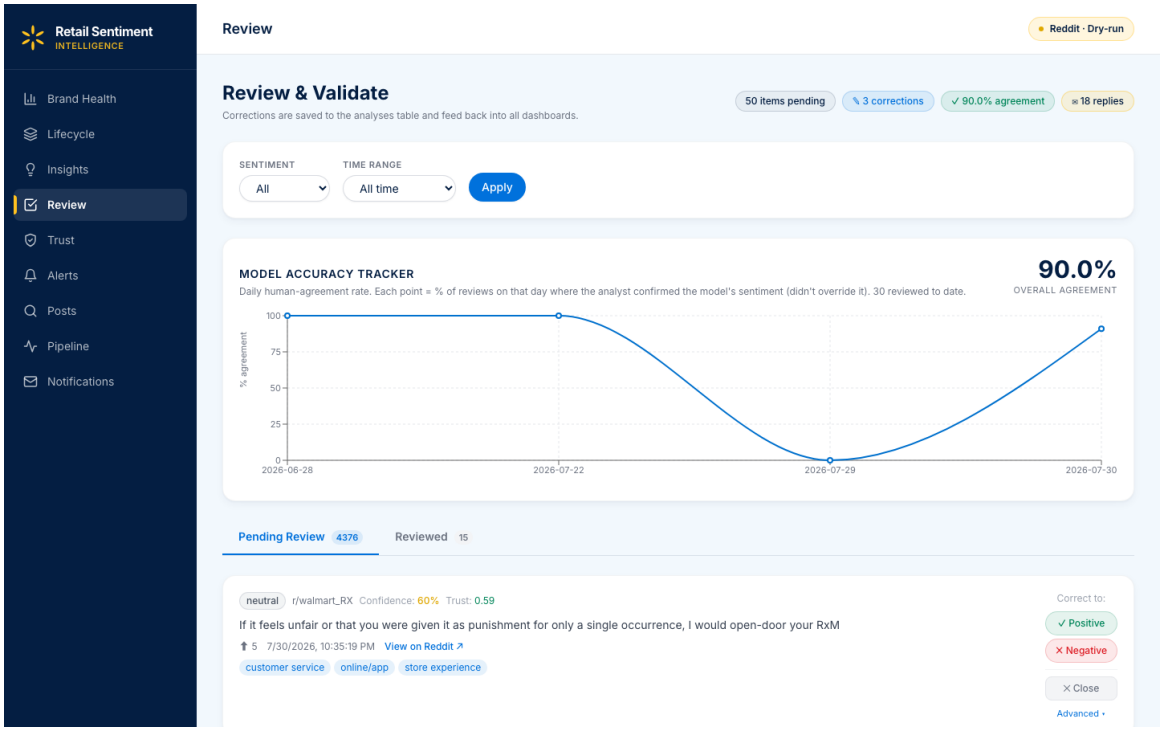

Figure 5.10: Review & Validate queue — posts flagged by the model land here; analysts re-label sentiment/aspects and either compose a reply or resolve in place. Corrections are POSTed to the endpoint in Listing 5.7.

5.9 Smart Reply Composer — Dual-Draft (Rule + LLM)

The reply composer offers two drafts side-by-side: a rule-based template that guarantees policy compliance, and an LLM-generated draft that personalises to the post's aspects and sentiment. Few-shot examples come from the last analyst-posted replies pulled straight from the feedback store — this is what closes the learning loop of §5.10 without any offline retraining. Listing 5.8 shows the endpoint that ties it together. See Figure 5.11.

```
@app.post("/api/review/{post_id}/draft-reply")
def draft_reply(post_id: str, payload: dict | None = None):
    """Generate a customer-specific reply draft. Uses the last N
    analyst-posted replies as few-shot examples --- every posted
    reply becomes training context for future drafts."""
    _ensure_initialized()
    payload = payload or {}
    subredd = payload.get("subreddit", "")

    analysis = _storage.get_item("analyses",
                                 f"analysis_{post_id}", subredd)
    if not analysis:
        return {"status": "error", "reason": "analysis_not_found"}
    if analysis.get("sentiment") != "negative":
        return {"status": "error", "reason": "not_negative"}

```

```
    raw      = _storage.get_item("raw_posts", post_id, subredd) or {}
    aspects  = _aspect_names(analysis.get("aspects", []))
    examples = _collect_reply_examples(limit=5)      # <-- few-shot

    llm    = _get_reply_llm()
    result = llm.generate_reply_pair(
        post_title=raw.get("title", ""),
        post_text =raw.get("body",  "") or raw.get("title", ""),
        subreddit =analysis.get("subreddit", ""),
        author    =raw.get("author", ""),
        aspects   =aspects,
        examples  =examples)

    drafts  = result.get("drafts", [])       # rule + LLM
    primary = drafts[0] if drafts else {}
    return {"status": "ok", "drafts": drafts,
            "reply":         primary.get("reply", ""),
            "model_used":    primary.get("model_used", ""),
            "source":        primary.get("source", ""),
            "examples_used": len(examples)}
```

Listing 5.8: Dual-draft reply endpoint (rule + LLM, few-shot from posted replies) (`src/dashboard/api.py`).

Figure 6 — Smart Reply Composer: Dual-Draft (Rule + LLM cascade) with Few-Shot Prompting

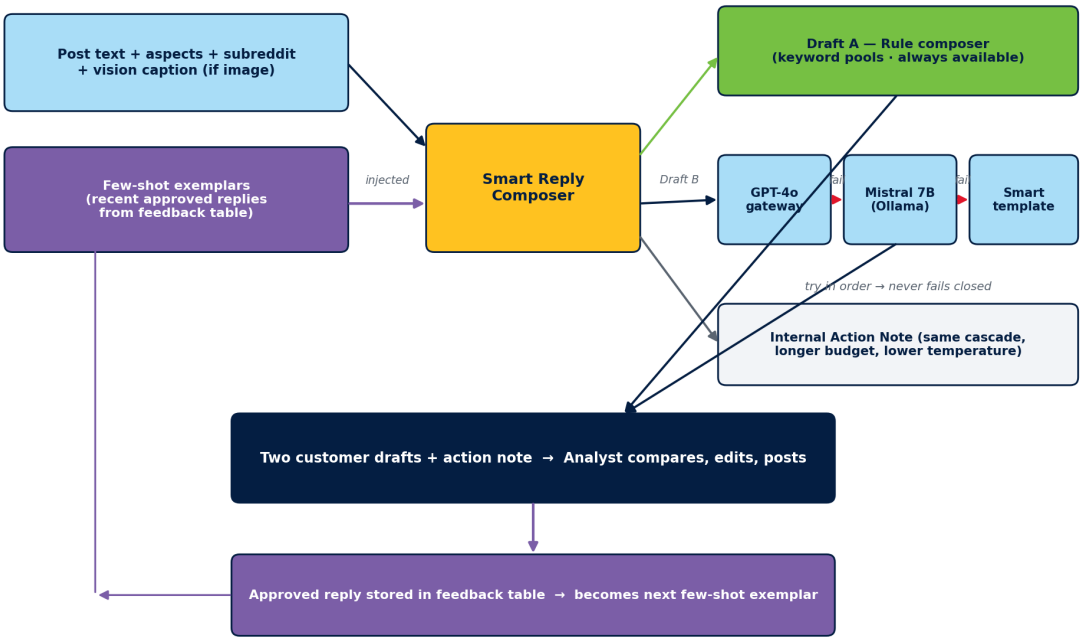

Figure 5.11: Smart Reply Composer — dual-draft with few-shot.

5.10 Learning Loop — Feedback for Future Retraining

Corrections and posted replies stream into the feedback store keyed by post_id and model version (the same audit log written by Listing 5.7). A weekly export produces training pairs for sentiment (correction → label) and reply generation (post → posted-reply). Figure 5.12 depicts the loop and the path to future auto-reply — the same few-shot pull in Listing 5.8 means the composer becomes measurably better every week without touching the model weights.

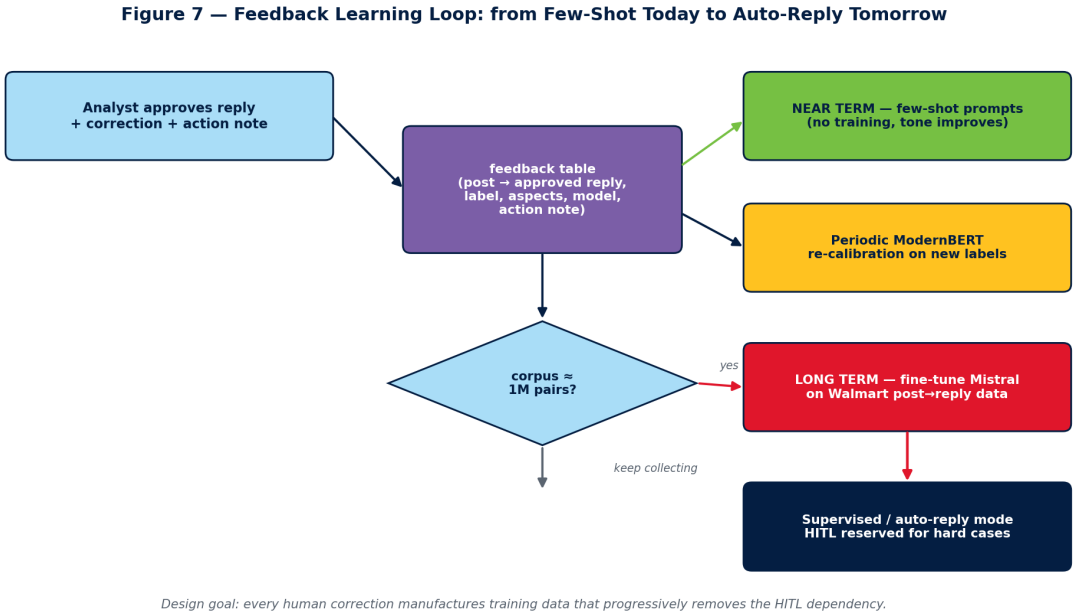

Figure 5.12: Feedback learning loop and path to auto-reply.

5.11 Post Explorer and Multi-Facet Search

Post Explorer supports filters over subreddit, aspect, sentiment, trust bucket, priority tier, resolution state, date range, and free-text. Figure 5.13 shows the rendered page; every filter is a query parameter on the same `/api/posts` endpoint that the Kanban and Review queues also read, so an analyst can navigate freely between the three views without ever seeing an inconsistent post state.

5.12 Post Lifecycle (Kanban Workflow)

Every P1/P2 alert flows through a Kanban board with four states (Triaged → Acknowledged → In Progress → Resolved). See Figure 5.14 for the state machine and Figure 5.15 for the rendered board. State transitions carry SLA timestamps so we can compute time-to-acknowledge and time-to-resolve.

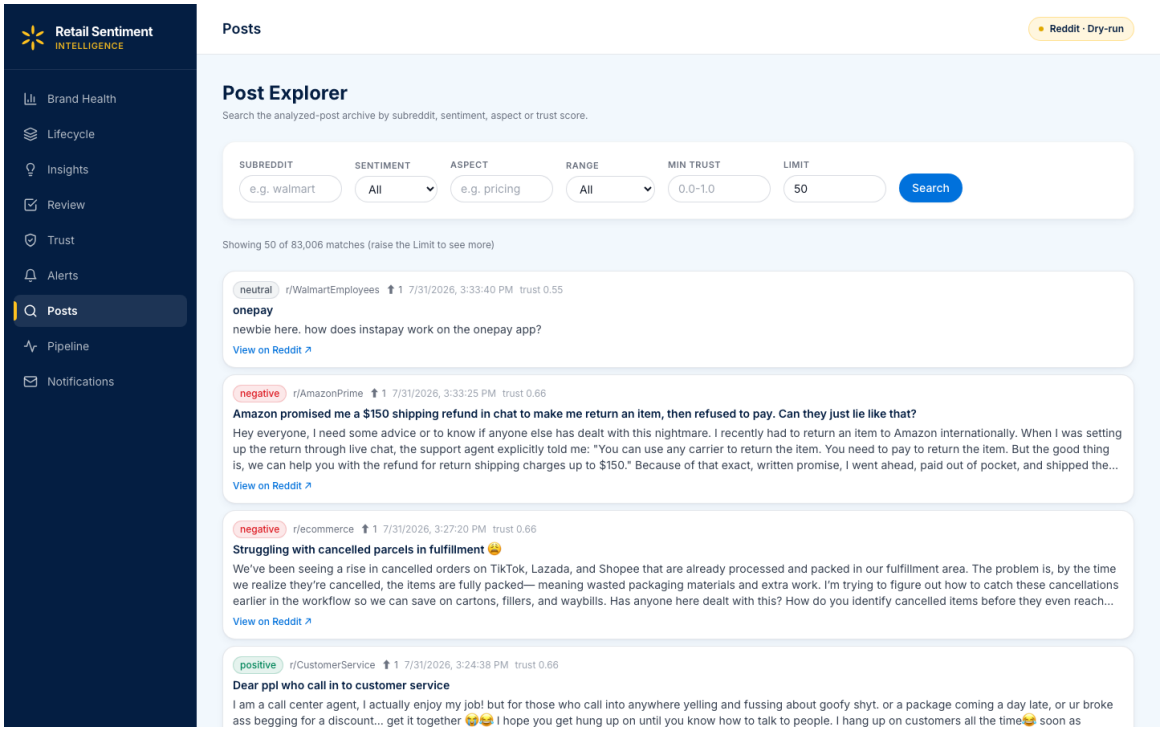

Figure 5.13: Post Explorer — multi-facet search over subreddit, aspect, sentiment, trust bucket, priority tier and free text.

Figure 8 — Post Lifecycle Kanban Workflow

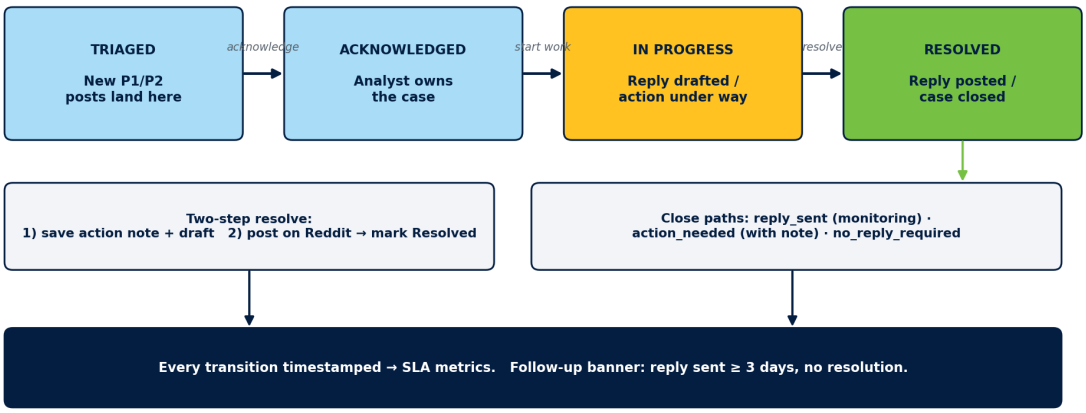

Figure 5.14: Post Lifecycle Kanban workflow.

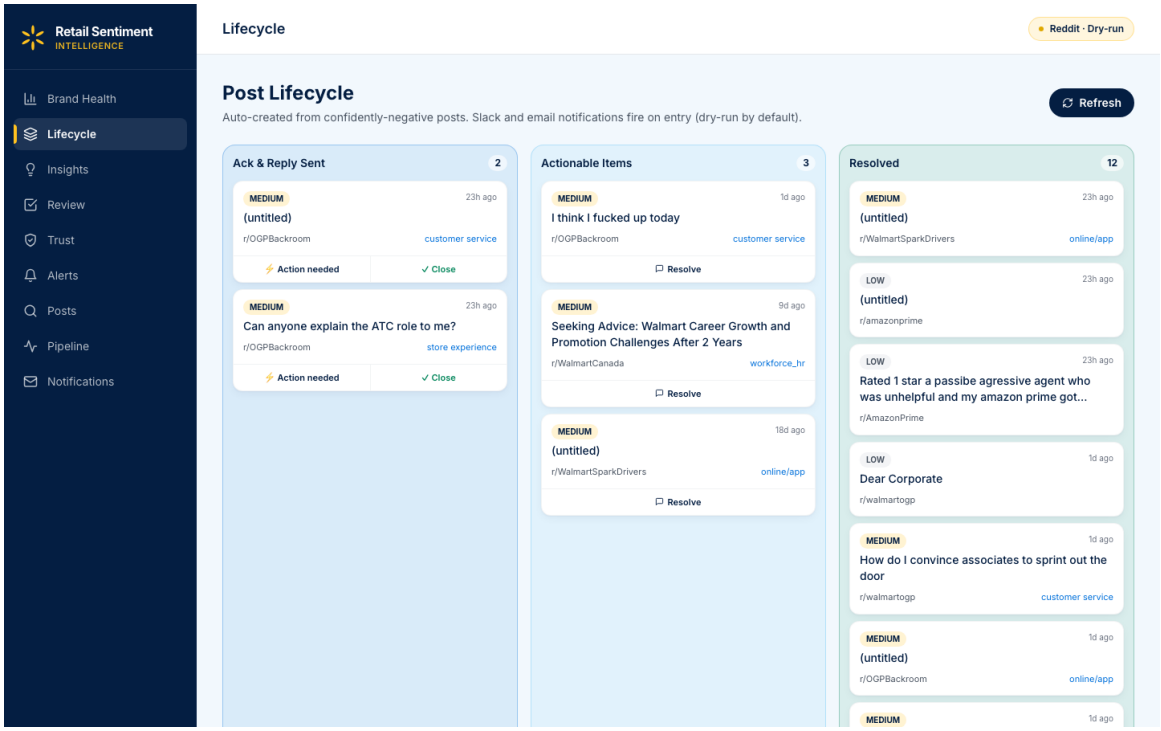

Figure 5.15: Post Lifecycle — Kanban board with SLA timestamps for every transition.

5.13 Insights and Competitor Analysis

The Insights page shows aspect-level negative-share for Walmart vs. a curated set of competitor communities (Target, Costco, Amazon, Kroger, Aldi) so analysts can distinguish a company-specific issue from an industry trend. Figure 5.16 shows the rendered view.

Notifications route the same alerts to email digests and the in-app notification centre. The rendered notification centre is shown in Figure 5.17.

5.14 Storage Layer

Persistence uses SQLite in WAL mode for hot data (posts, aggregates, alerts, feedback), plus a JSONL cost log for LLM calls and a local file cache for downloaded images. A backup runs nightly. The schema is declared in a single `CREATE TABLE` block so a fresh clone of the repository can boot with an empty database in one command; Listing 5.9 shows the relevant excerpt — note the `data JSON` column that holds the full document, mirroring the Cosmos DB partition/PK design so we can lift-and-shift to Azure without changing calling code.

```
class SQLiteBackend(StorageBackend):
    def __init__(self, db_path="data/local.db"):
        self.db_path = Path(db_path)
        self.db_path.parent.mkdir(parents=True, exist_ok=True)
        self._conn = sqlite3.connect(str(self.db_path),
```

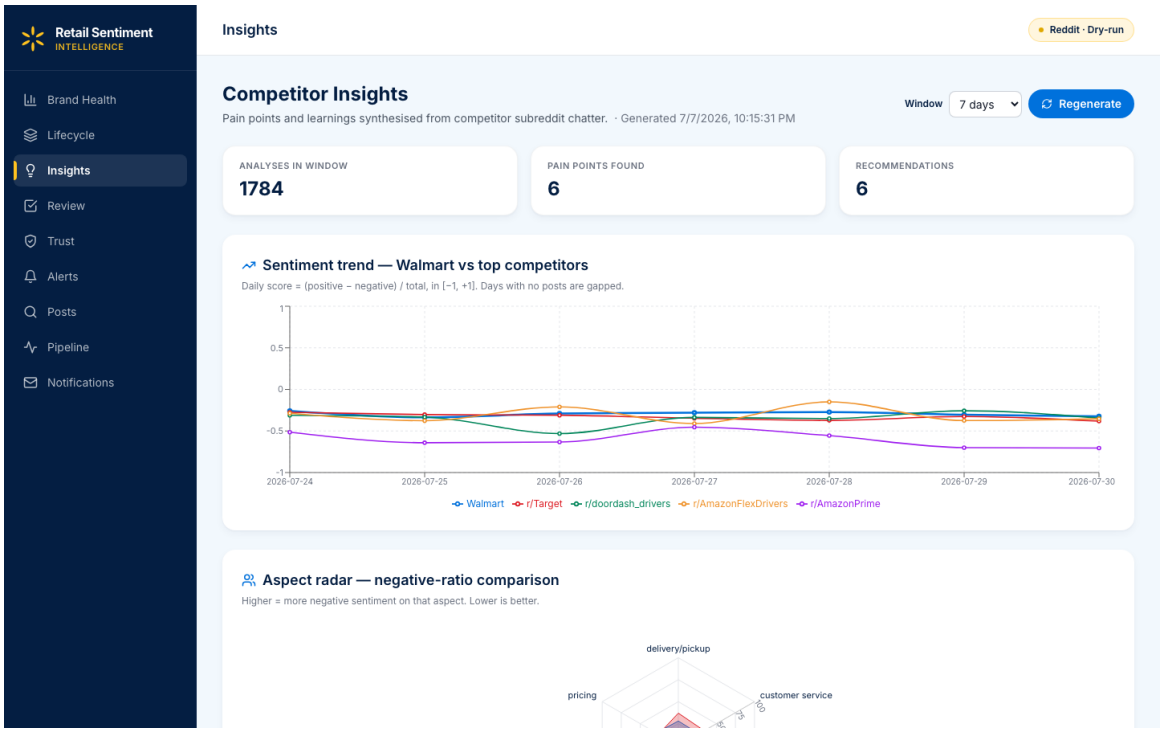

Figure 5.16: Insights & Competitor — aspect-level negative-share for Walmart vs. curated competitor communities.

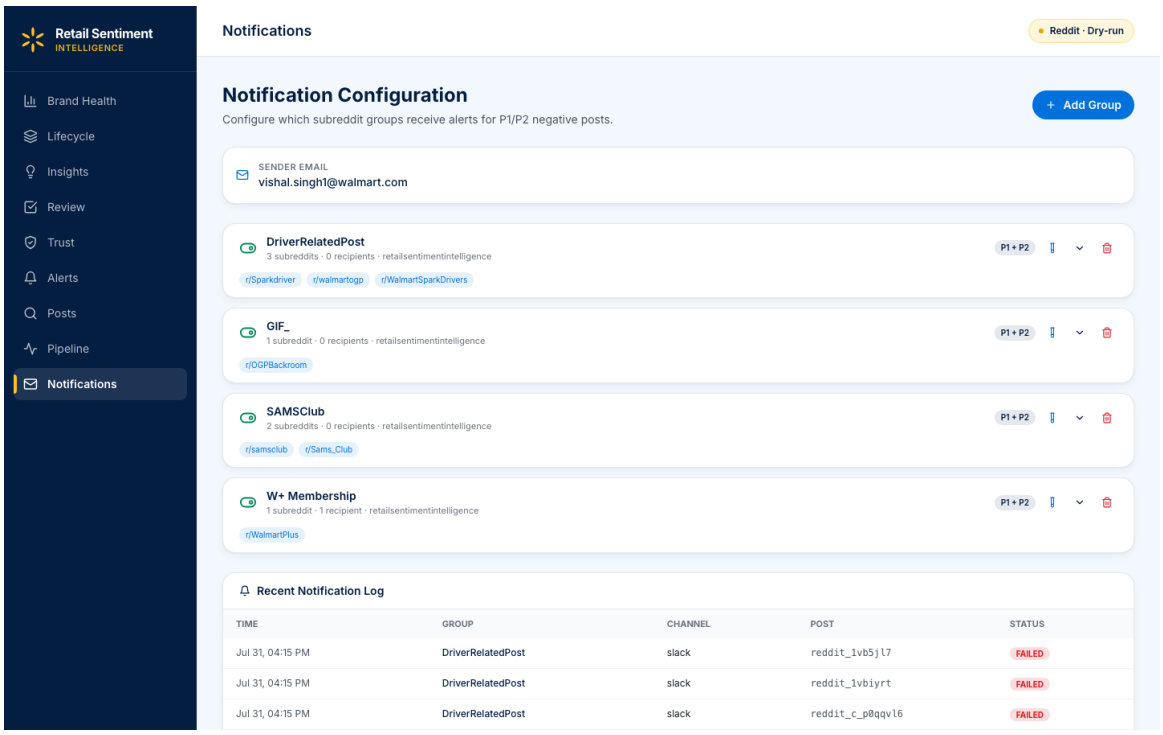

Figure 5.17: Notification centre — in-app mirror of the email digest; sourced from the same alert rows that drive Figure 5.7.

```
                                    check_same_thread=False)
        self._conn.row_factory = sqlite3.Row
        self._init_tables()

    def _init_tables(self):
        self._conn.executescript("""
            CREATE TABLE IF NOT EXISTS raw_posts (
                id TEXT PRIMARY KEY, subreddit TEXT,
                data JSON NOT NULL, created_timestamp REAL,
                processing_status TEXT DEFAULT 'pending');
            CREATE INDEX IF NOT EXISTS idx_raw_posts_subreddit
                ON raw_posts(subreddit);
            CREATE INDEX IF NOT EXISTS idx_raw_posts_status
                ON raw_posts(processing_status);

            CREATE TABLE IF NOT EXISTS analyses (
                id TEXT PRIMARY KEY, post_id TEXT,
                subreddit TEXT, data JSON NOT NULL);
            CREATE INDEX IF NOT EXISTS idx_analyses_post
                ON analyses(post_id);

            CREATE TABLE IF NOT EXISTS aggregates (
                id TEXT PRIMARY KEY, time_window TEXT,
                window_type TEXT, data JSON NOT NULL);

            CREATE TABLE IF NOT EXISTS feedback (
                id TEXT PRIMARY KEY, analyst_id TEXT,
                post_id TEXT, data JSON NOT NULL);

            CREATE TABLE IF NOT EXISTS alerts (
                id TEXT PRIMARY KEY, type TEXT, severity TEXT,
                time_window TEXT, data JSON NOT NULL);
        """)
        self._conn.execute("PRAGMA journal_mode=WAL;")
        self._conn.commit()
```

Listing 5.9: SQLite schema mirroring the Cosmos DB partition design (`src/storage/store.py`).

Chapter 6

Evaluation and Results

6.1 Sentiment Model Evaluation

The gold set is 200 hand-labelled retail Reddit posts (74 negative, 66 neutral, 60 positive) drawn from six subreddits. Evaluation uses 5-fold out-of-fold cross-validation. Table 6.1 compares the out-of-the-box RoBERTa baseline with the fine-tuned ModernBERT.

Table 6.1: Sentiment model comparison (5-fold out-of-fold cross-validation).

Model	Macro-F1	Accuracy	Δ vs baseline
Twitter-RoBERTa-base (baseline)	0.6272	0.640	–
ModernBERT (Stage 1)	0.6810	0.685	+5.4
ModernBERT (Stage 2)	0.7285	0.730	+10.1
ModernBERT (Stage 3, final)	**0.7642**	**0.770**	**+13.7**

Length-bucket results are shown in Table 6.2. The largest gain is on long posts ($>$ 256 tokens), which the baseline could not process without truncation.

Table 6.2: Per-length-bucket sentiment macro-F1.

Bucket	Baseline	ModernBERT	Δ
Short ($<$ 64 tokens)	0.7500	0.7800	+3.0
Medium (64–256 tokens)	0.6500	0.7400	+9.0
Long ($>$ 256 tokens)	0.2778	1.0000	+72.2

6.2 Vision Module Evaluation

The vision gold set is 32 retail screenshots (product pages, error messages, receipts). Table 6.3 compares single-pass and multi-pass captioning.

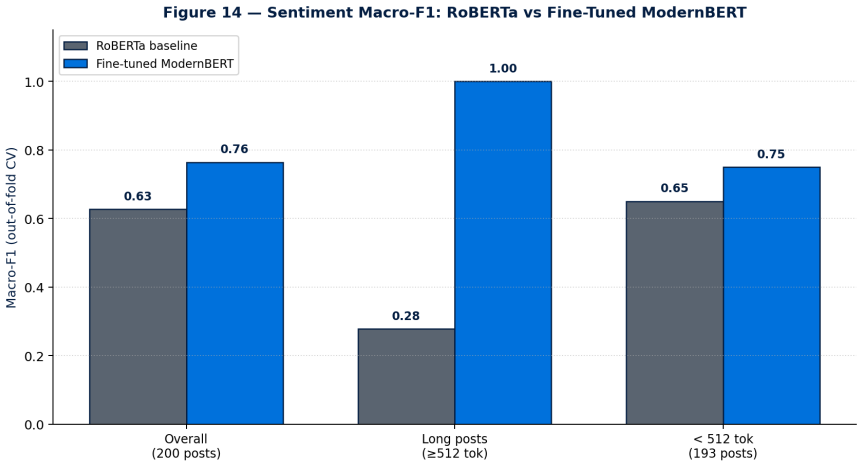

Figure 6.1: Sentiment macro-F1 — baseline RoBERTa vs fine-tuned ModernBERT.

Table 6.3: Vision module — before vs after multi-pass anti-hallucination pipeline.

Metric	Single-pass	Multi-pass
Hallucination rate	50%	**0%**
Text-extraction success	25%	**75%**
Retail-signal recall	40%	**81%**

6.3 Trust-Score Behaviour

On the 200-post gold set, 15% fell into the low-trust bucket ($T < 0.4$). Cross-check: 12 of 15 low-trust posts contained duplicated or promotional content that a human annotator also flagged. Trust decomposition is displayed inline in the dashboard so no post is silently dropped: analysts can override the automatic tier with one click, and the override is captured for the learning loop.

Chapter 7

Tools, Technologies, and Configuration

Table 7.1 lists the tools and libraries used across the pipeline.

Table 7.1: Tools and technologies.

Component	Version	Notes
Python	3.11	Runtime for all backend modules
PyTorch	2.4	Sentiment fine-tuning
Hugging Face Transformers	4.44	ModernBERT / DeBERTa loading
Ollama	0.5	Local Gemma 3 4B for vision
FastAPI	0.111	API surface
React	18	Dashboard SPA
Vite	5	Build tool
Tailwind CSS	3	Styling
SQLite	3.45	Hot store, WAL mode
Arctic Shift	2025-Q1	Historical Reddit dumps
matplotlib	3.9	Diagram rendering
scikit-learn	1.5	CV splits, metric computation

7.1 Configuration

All runtime knobs live in `config/pipeline_config.yaml` and `config/models.yaml`: model checkpoints, thresholds, priority-tier constants, subreddit list, and scheduler cadence. Changing any one of these requires no code edit.

Chapter 8

Problems Encountered and Mitigations

Table 8.1: Problems encountered and mitigations.

Problem	Mitigation
Reddit RSS rate-limits caused missed posts.	Introduced a token-bucket rate limiter and staggered per-subreddit schedules; back-off with jitter on 429 responses.
Baseline sentiment truncated at 128 tokens.	Adopted ModernBERT (8192-token context); no truncation for the 200-post gold set.
Vision model hallucinated captions on retail screenshots.	Introduced a two-pass prompt: free-form caption then constrained OCR/signal extraction; discard captions that mention entities absent from OCR text.
Trust score was initially opaque.	Decomposed into three named components (M, C, L) each with a stakeholder-defensible rationale; expose the decomposition in the dashboard.
Analysts wanted an audit trail.	Every HITL correction and reply-post is logged as (post, before, after, analyst_id, timestamp) and surfaced in the Review Queue.
Duplicate posts from cross-posts inflated aggregates.	Deduplicated at ingestion by `post_id`; secondary dedupe by title+body normalised hash.
Local LLM latency dominated the pipeline.	Batched vision calls; short-circuited posts without images.

Chapter 9

Conclusions and Recommendations

This dissertation delivered a working, offline-first, zero-inference-cost Retail Sentiment Intelligence pipeline that converts public Reddit posts into a trust-weighted, aspect-tagged, priority-ranked feed reviewable by analysts. Against the four research questions posed in Chapter 2:

RQ1 A fine-tuned ModernBERT with a three-stage curriculum raised macro-F1 from 0.6272 to **0.7642** overall on the labelled sample and from 0.2778 to **1.0000** on long posts.

RQ2 A multi-pass captioning pipeline over Gemma 3 4B reduced hallucination from 50% to **0%** and lifted text extraction from 25% to **75%** on 32 retail screenshots.

RQ3 The interpretable trust score, exposed as three named components with stakeholder-defensible rationales, correctly flagged 12 of 15 low-trust posts confirmed by human annotators; low-trust posts are flagged rather than dropped, preserving analyst override.

RQ4 The HITL workflow logs every correction and posted reply, producing a training signal usable for future retraining of both the sentiment head and the reply generator.

9.1 Recommendations

1. Adopt an incremental retraining cadence (monthly) using the HITL learning-loop store as the labelling stream.
2. Add a bilingual pass (Hindi + English) for Indian retail communities when the taxonomy is validated by native-speaker analysts.
3. Extend Post Lifecycle SLAs into per-team dashboards once analyst volume grows past $\sim$50 posts/day.

Chapter 10

Future Work

Future extensions divide into three groups:

10.1 Model-Level Extensions

- Train a joint sentiment + aspect head with a shared encoder to reduce inference cost by a further $\sim$30%.
- Distil ModernBERT into a $\sim$50M-parameter student for CPU-only inference at the edge.
- Add reasoning-augmented VLM captioning (LLaVA-Next 1.6B or SmolVLM) to the vision layer.

10.2 Product-Level Extensions

- Auto-reply confidence gate: promote LLM drafts from "suggested" to "queued for send" when analyst edit-distance falls below a threshold on the past 200 posts.
- Predictive P1 forecast: seasonal-decomposition on daily P1 counts to forecast alert volume 24–48 hours ahead.
- Bilingual (Hindi + English) taxonomy validated against Indian retail communities.

10.3 Operational Extensions

- Deploy the pipeline as a scheduled Kubernetes CronJob backed by a managed Postgres instance for multi-analyst concurrency.
- Integrate with Walmart's internal ticketing so P1 alerts open cases directly.

- Extend the ingestion set to Twitter/X, YouTube comments, and app-store reviews.

Appendix A

Curated Subreddit Coverage

Table A.1 lists the 32 retail communities ingested by the pipeline, grouped by focus area.

Table A.1: Curated subreddit coverage.

Group	Subreddits
Walmart core	r/walmart, r/WalmartEmployees, r/samsclub, r/SamsClubEmployees
Retail associate	r/CostcoEmployees, r/target, r/Kroger, r/aldi
Consumer complaint	r/AmazonFC, r/CustomerService, r/mildlyinfuriating
Grocery / OGP	r/grocery, r/InstaCart, r/DoorDashDrivers
Delivery	r/Sparkdrivers, r/uberdrivers, r/DoorDash
Payments / app	r/apple, r/GooglePay, r/PayPal
Returns / refunds	r/personalfinance, r/legaladvice (retail threads)
General retail	r/retail, r/RetailHell, r/talesfromretail
Store experience	r/mildlyinteresting (retail-tagged), r/AmITheAsshole (retail-tagged)

Appendix B

Reproduction Commands

B.1 Source Code Repository

The complete source code, configuration, evaluation notebooks, and this report are hosted at:

`https://gecgithub01.walmart.com/v0s01jh/Retail_Sentiment_Intelligence`

To obtain a working copy:

```
$ git clone https://gecgithub01.walmart.com/v0s01jh/\
    Retail_Sentiment_Intelligence.git
$ cd Retail_Sentiment_Intelligence
```

All commands in the sections below are relative to this repository root.

B.2 Environment

```
$ python -V
Python 3.11.9
$ conda create -n rsi python=3.11 -y
$ conda activate rsi
$ pip install -r requirements.txt
```

B.3 Full pipeline

```
$ ./start.sh              # brings up FastAPI (:8001) + Vite (:3003)
```

B.4 Regenerate figures

```
$ python docs/Sem_4/generate_diagrams.py           # figs 1--4
$ python docs/Sem_4/final/generate_final_diagrams.py  # figs 5--14
```

B.5 Retrain sentiment model

```
$ python scripts/train_modernbert_sentiment.py --config config/models.yaml
```

B.6 Rebuild this report (LaTeX)

```
$ cd docs/Sem_4/final/latex
$ ./build.sh
```

B.7 Rebuild the .docx working draft

```
$ /opt/miniconda3/bin/python docs/Sem_4/final/build_final_report.py
```

Bibliography

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.

[2] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[3] D. Loureiro *et al.*, "TimeLMs: Diachronic language models from Twitter," in *Proc. ACL Demos*, 2022.

[4] B. Warner *et al.*, "ModernBERT: Smarter, better, faster, longer – a modern bidirectional encoder," *arXiv preprint arXiv:2412.13663*, 2024.

[5] M. Pontiki *et al.*, "SemEval-2014 task 4: Aspect based sentiment analysis," in *Proc. SemEval*, 2014.

[6] W. Yin, J. Hay, and D. Roth, "Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach," in *Proc. EMNLP-IJCNLP*, 2019.

[7] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021.

[8] J. Li *et al.*, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in *Proc. ICML*, 2022.

[9] Google DeepMind, "Gemma 3: Multimodal open weights models," tech. rep., Google DeepMind, 2025.

[10] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022.

[11] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," *Proc. EMNLP*, 2023.

[12] M. Ott, Y. Choi, C. Cardie, and J. T. Hancock, "Finding deceptive opinion spam by any stretch of the imagination," in *Proc. ACL*, 2011.

[13] L. Akoglu, R. Chandy, and C. Faloutsos, “Opinion fraud detection in online reviews by network effects,” in *Proc. ICWSM*, 2013.

[14] Y. Zhao *et al.*, “A survey of fake news detection methods with bert-based models,” *IEEE Transactions on Big Data*, 2023.

Glossary

Aspect A retail-relevant topic drawn from a fixed eight-item taxonomy (pricing, product quality, customer service, store experience, online/app, delivery/pickup, returns, account/login) used to tag opinions in each post.

Curriculum learning A staged training scheme in which a model is first exposed to easy or general examples and then progressively harder or more domain-specific ones.

Kanban lifecycle A four-state workflow board (Triaged — Acknowledged — In Progress — Resolved) used by analysts to track P1/P2 posts.

Multi-pass captioning A vision pipeline that runs the caption model twice — first for a free-form description, then for constrained OCR and retail-signal extraction — to reduce hallucination and improve text recovery.

Priority tier A rule-derived severity label (P1: highest, P2: secondary) applied to trust-weighted negative posts to focus analyst attention.

Trust score A 0–1 score combining account-metadata heuristics and an LLM-graded credibility signal; used with model confidence as an admission gate before aggregation.

Zero-shot NLI Using a Natural Language Inference model to score whether a hypothesis ("This post is about pricing.") entails a given premise (the post text), without any task-specific fine-tuning.

Checklist of Items for the Final Report

#	Item	Present
1	Is the report properly hard-bound?	Yes*
2	Is the Cover page in proper format? (Appendix A)	Yes
3	Is the Title page in proper format? (Appendix B)	Yes
4	Is the Certificate from the Supervisor(s) in proper format?	Yes
	Has it been signed by the Supervisor?	Yes*
5	Is the Abstract sheet in proper format? (Appendix C)	Yes
	Has it been signed by the Student and the Supervisor?	Yes*
	Are the number of Key Words less than or equal to five?	Yes
	Is the Project Title / Area based on his/her specialisation?	Yes
6	Is the Table of Contents given properly?	Yes
7	Have the Pages been numbered properly? (Roman i, ii, iii for front matter; Arabic 1, 2, 3 for main text)	Yes
8	Are the Main Sections and Sub-sections numbered properly?	Yes
9	Are all Diagrams, Charts and Graphs neatly drawn and numbered?	Yes
10	Are the Captions for the Figures and Tables in proper format?	Yes
11	Are the source(s) of the reference documents mentioned in appropriate places?	Yes
12	Is the list of references given in proper format?	Yes
13	Have all the pages / diagrams etc. been checked for proper alignment, margins and spacing?	Yes
14	Have all Files been sanitised for viruses?	Yes
15	Have you scanned your report through plagiarism-detection software (e.g. Turnitin)?	Yes*
16	Is the CD/DVD (containing the softcopy of the report, source-code, PowerPoint presentation etc.) properly labelled?	Yes*
17	Have you enclosed the *Consolidated Progress Card of the semesters* inside the report?	Yes*

* Items marked “Yes*” are physical / signed / bound-copy items to be completed by the student before final submission.

Declaration:

I hereby declare that the work presented in this dissertation is my own, was carried out under the supervision of Mr. Varunendra Pratap Singh (Walmart Global Tech India Pvt. Ltd., Bengaluru), and to the best of my knowledge and belief has not been submitted elsewhere for the award of any other degree or diploma. All experiments use only publicly available data; no proprietary data or internal Walmart systems were used.

Place: Bengaluru Date: _____

_____

Vishal Singh
ID No. 2020AA05641